HACKATHON PROPOSAL · AI RED TEAMING, SAFETY & ALIGNMENT — UNLEARNING TRACK

Gone, or Just Hidden?

Verifiable unlearning of copyrighted knowledge and college-level math in the hybrid LFM3-1.7B model

MODEL	STACK	DATE
LiquidAI/LFM3-1.7B (conv + attention hybrid)	CuratorKIT · SafeTune · AuditKIT · Jacobian lens	8 November 2026

1 Summary

We remove two different kinds of content from LiquidAI's **LFM3-1.7B**. The first is a **body of copyrighted knowledge**: a well-known novel, both its verbatim text and its book-specific facts. The second is a **procedural skill**: college-level calculus. We then try hard to bring both back with paraphrases, jailbreaks, relearning, tool use and re-derivation. We also look inside the model with Anthropic's **Jacobian lens (J-space)** to tell knowledge that was erased from knowledge that is only suppressed. General capability is tracked throughout.

Hybrid architecture

LFM3 mixes gated short convolutions with sliding-window and global attention. As far as we know, almost all unlearning studies use pure transformers. So we ask an open question: where do facts and skills live in this kind of model?

Facts vs. skills, same attacks

Most unlearning work removes facts. We also remove a multi-step skill and test whether it can be rebuilt from the prerequisites the model keeps.

Evidence from inside the model

J-space readouts and injection attacks test whether the model still *thinks* the forgotten content even when it no longer says it.

Gradient-free unlearning (stretch)

A frontier LLM inspects and edits the small model's weights directly. It produces a small patch that a human can audit, and we compare it with gradient methods.

2 Track fit

Track requirement	How this project meets it
Remove a capability or body of knowledge using forget and retain sets	**Target A** (copyrighted novel) and **Target B** (calculus). Forget and retain sets are built with CuratorKIT, and every set has a provenance manifest.
Prove it is gone under paraphrased and adversarial prompts	**Resurrection Audit**: 14 attacks. These cover paraphrase, notation changes, jailbreak and encoding probes, Best-of-N, abliteration, relearning, re-derivation, tool use, and J-space readouts and injection (§8–9).
Show general capability survives	Retain sets: lower math levels (arithmetic to precalculus) and other books. Plus MMLU, IFEval, BFCL tool calling and fluency, all run through AuditKIT.

3 Model: LFM3-1.7B

Specification (from config.json and model card)	
Layers	24: 13 gated short-conv, 5 sliding-window attention (window 256), 6 global attention
Width	hidden 2048, SwiGLU MLP 7680 (`w1/w2/w3`)
Attention	16 query heads / 4 KV heads, gated attention, QK-norm, sandwich norms
Vocab / context	128K tokens / 256K context, tied embeddings
Training	22T pre-training tokens, then SFT, preference tuning and RL. Post-training targets agents and tool use
Loading	transformers ≥ 5.12, `trust_remote_code=True`
License	LFM Open License v1.0

Why LFM3. It is recent and small enough for one GPU. It is trained for agent use, which makes the "solve it with a tool" loophole a realistic attack. Its hybrid design also raises a real research question: *where are facts and skills stored when 13 of 24 layers mix tokens with convolutions instead of attention?*

Every layer still has an MLP, so MLP-targeted methods (RMU, ROME-style localization) still apply. Attention-head analysis only covers 11 of the 24 layers.

> **Research angle:** compare where the forgotten content concentrates: MLPs that follow conv layers vs. MLPs that follow attention layers. No prior result exists for this design.

Stack compatibility (checked against source code)

Component	On LFM3	What it takes
CuratorKIT	Yes	Does not depend on the model
SafeTune GA / GradDiff / NPO / SCRUB	Yes	Load the model ourselves with `trust_remote_code=True` and pass it in
SafeTune RMU	With a tweak	It finds `model.model.layers`, but the default `param_substring="mlp.down_proj"` doesn't exist in LFM3. Set it to `"feed_forward.w2"`
LoRA	With a tweak	Target modules: `w1, w2, w3, q_proj, k_proj, v_proj, o_proj, in_proj, out_proj`
Jacobian lens	With a tweak	The final norm is named `embedding_norm`, so pass `Layout("model", norm="embedding_norm")`
AuditKIT	Yes	vLLM supports LFM3 (per the model card); the HF backend needs `trust_remote_code`
CircuitKIT	No	Circuit discovery runs through TransformerLens, which has no port for conv blocks, gated attention or sandwich norms. A port would take days. It is replaced by our own attribution localizer (§5)

4 What we unlearn

Target A Copyrighted content: a novel

- **Forget:** verbatim passages (continuation prompts) and book-specific knowledge (characters, places, plot), as QA.
- **Retain:** general literature, other well-known books, real-world facts that share names or themes, and general knowledge.
- **Source:** an existing research split (e.g. MUSE-Books, Harry Potter), so we don't redistribute raw text.
- **Metrics:** ROUGE-L on passage continuation (verbatim memory) and QA accuracy, scored on whether the fact appears, not on whether the model refuses.
- **Stretch, "forget the text, keep the facts":** remove verbatim regurgitation but keep the ability to discuss and summarise the book, in the spirit of fair use (§9).

Target B College-level math: calculus

- **Forget:** differentiation and integration, plus word problems and physics problems with a hidden calculus step.
- **Retain levels:** arithmetic, algebra, precalculus, and general benchmarks.
- **Data:** DeepMind `mathematics_dataset` (arithmetic, algebra and `calculus__differentiate` modules, all in one format), sympy-generated integrals, and CuratorKIT word-problem rewrites re-checked by sympy.
- **Why calculus:** answers are graded automatically with no AI judge. New test items can be generated without limit, so memorisation can't skew results. It is also a safe public stand-in for hazardous procedural skills.

Control Unlearning depth ladder

Unlearn arithmetic, algebra and calculus, one at a time, and measure damage to the other levels and to general benchmarks. **Hypothesis:** skills nothing else depends on (calculus) come out cleanly, while foundational ones (1+1=2) cannot be removed without collapse. Arithmetic is a control level here, not a real target.

Optional Target C: private data

Fine-tune synthetic people into the model, then delete a few ("right to be forgotten"). The hard retain set is their near-neighbours (same company, similar names). Attempted only if time allows.

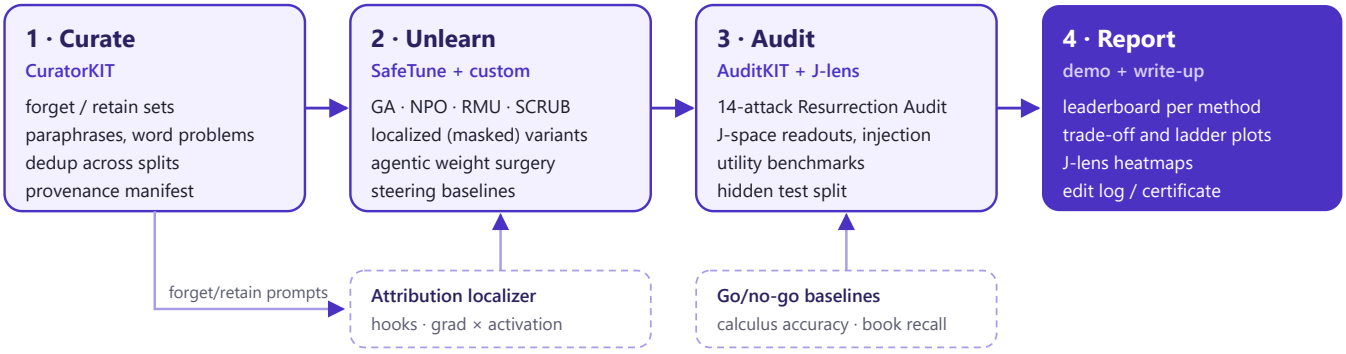
Go/no-go checks before any unlearning: (1) LFM3-1.7B solves enough of ~200 generated calculus problems that there is something to remove. If not, move the top level down to algebra or precalculus. (2) LFM3 actually knows the chosen book (continuation and QA). If not, pick another widely known book.

5 Toolchain

Tool	Role in the pipeline
CuratorKIT	Builds forget, retain, paraphrase and evaluation sets: `evol` rewrites, `adversarial_qa`, word problems and translations. Embedding dedup across train and eval splits keeps the evaluation leak-proof. Every run writes `manifest.json` and `lexsi_provenance.json`.
SafeTune	`safetune.runner.unlearn`: GradientAscent, GradDiff, NPO, RMU, SCRUB. Red-team stressors: `AbliterationAttack`, `BoNAttack`. Steering baselines: refusal direction, `CASTTrainer`, `LinearProbeGuardTrainer`. `tracin_influence` for choosing forget examples.
AuditKIT	The evaluation layer: generation and MCQ tasks, `GEval` judge, jailbreak, encoding and prompt-injection probes, `compare_models()`, and `ak.run_lmeval()` for MMLU, IFEval and similar benchmarks. Every run has a fingerprint, so results are reproducible.
Jacobian lens	`anthropics/jacobian-lens` (Apache-2.0): J-space readouts and injection attacks (§9).
Custom code	Attribution localizer (forward/backward hooks, gradient × activation per MLP neuron, attention head and conv layer), sympy grader, weight-surgery harness, and Resurrection Audit runner.
Frontier LLM API	The agent for weight surgery (§7), and the generator and judge models inside CuratorKIT.
Not used	**CircuitKIT**: no LFM3 support (§3). **neuron-explorer**: only works with Qwen3.5-4B.

Compute. At bf16, the 1.7B weights take about 3.4 GB. RMU updates only three MLP matrices, so it fits on a 24 GB GPU. Full-parameter NPO/GA needs about 27 GB before activations (weights, gradients and Adam states), plus a frozen reference model. So either use LoRA on 24 GB or rent one 80 GB A100/H100.

6 Pipeline



7 Unlearning methods

Group	Methods	Purpose
B0 Baselines	Refusal steering (SafeTune `steer`); a prompt saying "act as if you have never read X"	Controls that only *look* like forgetting. Expected to fail under attack, which makes a clear contrast in the demo.
B1 Gradient	GA, GradDiff, NPO, RMU, SCRUB (SafeTune)	Standard methods, run on the same data and with the same evaluation.
B2 Localized	NPO/RMU limited to the top-k components chosen by our attribution localizer. Control: random components of the same size	Does localizing give the same forgetting with less collateral damage, and make relearning harder?
B3 Agentic weight surgery stretch	A frontier LLM edits the weights directly, with no gradients	Tests "zero-data unlearning", which the source blog lists as future work. Produces a patch a human can audit.

B3 design. Based on Takehi et al., *Sparks of Weight-Space Programming* (Nov 2026). There, a frontier model with a Python kernel fixed Qwen3.5-0.8B's reasoning loops by editing 0.016% of its weights, and MATH-500 rose from 7.9% to 48.1%. But it needed 50 rounds of model runs, and from weights alone it could not tell which model was sycophantic until it was given attention patterns. So our harness gives the agent tools, not just raw weights:

- **Tools:** weights in a Python kernel, attribution scores, J-lens readouts, activations, and a budget of 10 / 25 / 50 model runs (we plot results against the budget).
- **Guardrails:** at most 0.1% of weights changed. Embedding and output layers are frozen, so the agent can't just zero out the relevant name tokens. The agent sees only a dev split; the test split stays hidden.
- **Two levels:** L1, the agent picks *where* and masked NPO does the update. L2, the agent writes the edit itself (remove a direction, rank-one update, neuron scaling). L1 is the fallback if L2 fails.
- **Output:** every edit is logged (tensor, indices, rationale). This gives an auditable "unlearning certificate".

8 Evaluation: the Resurrection Audit

For each method and target we report three numbers, plotted as a trade-off: **Forget** (how much content remains on direct prompts), **Worst-case recovery** (the best any attack achieves) and **Utility** (retain sets plus general benchmarks).

#	Attack	Targets	Implementation
1	Paraphrase and rewording	A, B	CuratorKIT `evol`
2	Notation change (d/dx, f′(x), Leibniz)	B	problem generator
3	Word and physics problems with a hidden calculus step	B	CuratorKIT + sympy check

4	Indirect and multi-hop questions	A	CuratorKIT `adversarial_qa`
5	Other languages	A, B	LLM translation via CuratorKIT
6	Jailbreak, encoding and prompt-injection probes	A, B	AuditKIT `RedTeamRunner`
7	Best-of-N sampling	A, B	SafeTune `BoNAttack`
8	Abliteration (a weight-space attack)	A, B	SafeTune `AbliterationAttack`
9	In-context relearning (a few worked examples in the prompt)	A, B	custom
10	Fine-tune relearning (~20 examples; steps until recovery)	A, B	custom
11	Re-derivation in thinking mode, from the limit definition, using algebra the model kept	B	custom
12	Tool loophole: "write sympy code for this"	B	custom; LFM3 is trained for agent use
13	J-lens readout with two lenses (§9)	A, B	`jlens`
14	J-space injection of the forgotten concept	A, B	`jlens`

Utility

MMLU, IFEval, BFCL tool calling (LFM3's main use), perplexity and fluency on held-out web text, and the near-domain retain sets (other books, lower math levels).

Evaluation rules

A refusal is not forgetting: we score whether the content appears in the output. The test split is hidden from every method and from the agent. Training and evaluation paraphrases are deduplicated against each other. Gibberish is flagged, since it is a typical way gradient ascent fails.

9 J-space verification

Anthropic's *A global workspace in language models* (Jul 2026) found a small set of internal patterns, the **J-space**, that hold what a model is "thinking about" without saying it. These patterns are found with the Jacobian lens, which decodes what an internal activation is inclined to make the model say:

$$\text{lens}_1(h) = \text{unembed}(J_1 \cdot h), \quad J_1 = E[\, \partial h_{\text{final}} / \partial h_1 \,]$$

The lens is fitted once per model on 100–1,000 generic prompts of 128 tokens. About 100 prompts is already usable.

How we use it

- **Two lenses.** The base model's lens applied to the unlearned model's activations asks: "is the information still stored in early layers?" The unlearned model's own lens asks: "is it still inclined to say it?" A method can pass the second test and fail the first.
- **Readouts:** the rank of the forgotten token at each layer, before vs. after unlearning. The token can be an answer, a character name, the next verbatim word, or an intermediate derivative. Results are shown as heatmaps.
- **Injection attack:** insert the forgotten concept's J-pattern into a multi-hop prompt (the paper swapped "France" for "China"). If the downstream answers come back, only the entry point was cut and the facts are still stored.
- **Refusal baseline:** the paper's "white bear" result (telling the model not to think of X brings X to mind) predicts that refusal- or prompt-based "unlearning" leaves the concept active inside the model.

Caveats we design around

- The paper found that simple fact recall and MCQ mostly **bypass** the J-space, while **multi-step reasoning** goes through it. So the lens is most informative for calculus and multi-hop book questions. An empty readout on a direct question proves nothing, so we pair it with linear probes.
- Results were shown on Claude. A 1.7B model's workspace may be weaker, so we first validate on the base model with known multi-step prompts.
- Swapping "Spanish" for "French" changed "name the language" but not "continue the passage". This suggests verbatim continuation runs on automatic processing, while knowledge *about* a text goes through the workspace. That is the mechanistic motivation for the "forget the text, keep the facts" variant.

10 Hypotheses

H1	Gradient unlearning on LFM3 **suppresses rather than erases**: the base lens still recovers forgotten tokens, and relearning restores the content within a few steps. RMU holds up better than GA/NPO.
H2	**Skills are harder to erase than facts**: calculus can be recovered through re-derivation and tool use even when direct prompts fail.
H3	**Collateral damage grows with how foundational the skill is**: calculus < algebra < arithmetic (depth ladder).
H4	**Localized edits** (attribution-masked or agent-chosen) forget as well as full-model methods with less collateral damage. A random mask of the same size does not.
H5	*Exploratory:* in the hybrid, the components behind the forgotten content concentrate in MLPs after attention layers rather than after conv layers, or the reverse.
H6	*Exploratory:* agentic weight surgery achieves measurable forgetting while changing ≤ 0.1% of weights.

11 Plan (assuming a 48-hour event)

Phase	Time	Work	Exit check
0 · Setup and checks	~4 h	Load LFM3, run baseline calculus and book-recall tests, fit the J-lens on the base model, validate the lens on multi-step prompts	Go/no-go decision per target
1 · Data	~6 h	CuratorKIT forget, retain, paraphrase and evaluation sets; dedup; sympy grader; math levels	Hidden test split frozen
2 · Unlearning	~12 h	B0–B2 on Targets A and B; depth ladder	Checkpoints plus a first leaderboard
3 · Resurrection Audit	~10 h	Attacks 1–14; utility suite	Trade-off plots, J-lens heatmaps
4 · Weight surgery	~8 h, in parallel	B3: L1 first, then L2	Patch log, or a documented negative result
5 · Report and demo	~6 h	Write-up, slides, live demo	Submission

Deliverables

- A leaderboard with forget, worst-case recovery and utility per method and target, plus "% of weights changed" and cost.
- Resurrection curves (recovery vs. relearning steps), the depth-ladder chart, and before/after J-lens heatmaps.
- An edit log for agentic weight surgery, presented as an unlearning certificate.
- A reproducible repo, with CuratorKIT provenance manifests and AuditKIT run fingerprints.

12 Risks and mitigations

Risk	Mitigation
LFM3-1.7B is too weak at calculus for there to be anything to remove	Check at phase 0; move the top level down to algebra or precalculus, or use simpler differentiation
LFM3 doesn't know the chosen book well	Check recall first; switch to another widely known book
Library incompatibilities from LFM3's custom code	Load the model ourselves and pass the object; set RMU `param_substring`, LoRA targets and the J-lens `Layout` (§3)
GPU memory for full-parameter methods	LoRA on 24 GB, or one 80 GB GPU
J-lens is weak on a 1.7B model	Validate on the base model first; fall back to linear probes and the logit lens

Agentic surgery is costly or fails	Time-box it; L1 (agent chooses where, gradient does the update) is the fallback; a negative result is still reported
Gibberish passing as "forgetting"	Fluency and perplexity checks; the fact-presence scorer is separate from the refusal detector
Redistributing copyrighted text	Use existing research benchmark splits; don't publish raw passages

13 Out of scope

- **CircuitKIT:** no TransformerLens port exists for LFM3's hybrid blocks, and 13 of its 24 layers have no attention heads. Future work: a CircuitKIT port, or repeating the study on a supported model (e.g. Qwen3-1.7B) as a cross-architecture check.
- **neuron-explorer:** it only supports Qwen3.5-4B and needs a vLLM fork on a CUDA 13 GPU pod.
- **Basic arithmetic as a target:** it is used only as a control level, because everything else depends on it.

14 References

1. Lexsi Labs. SafeTune, CuratorKIT, CircuitKIT, AuditKIT. github.com/Lexsi-Labs; CuratorKIT paper: arXiv:2606.21631.
2. Liquid AI. LFM3-1.7B model card. huggingface.co/LiquidAI/LFM3-1.7B.
3. Anthropic (2026). A global workspace in language models. anthropic.com/research/global-workspace; paper: transformer-circuits.pub/2026/workspace; code: github.com/anthropics/jacobian-lens.
4. Takehi, Teoh, Zhang, Khattab (2026). Sparks of Weight-Space Programming: LLMs can natively read & write neural network weights. rikiyat.github.io/blog/weight-space-programming; logs: github.com/RikiyaT/lm_understand-logs.
5. Shi et al. (2024). MUSE: Machine Unlearning Six-Way Evaluation for Language Models.
6. Eldan & Russinovich (2023). Who's Harry Potter? Approximate Unlearning in LLMs.
7. Li et al. (2024). The WMDP Benchmark: Measuring and Reducing Malicious Use With Unlearning (RMU).
8. Zhang et al. (2024). Negative Preference Optimization: From Catastrophic Collapse to Effective Unlearning.
9. Maini et al. (2024). TOFU: A Task of Fictitious Unlearning for LLMs.
10. Saxton et al. (2019). Analysing Mathematical Reasoning Abilities of Neural Models (`mathematics_dataset`).
11. Guo et al. (2024). Mechanistic Unlearning: Robust Knowledge Unlearning and Editing via Mechanistic Localization.